Party Feature Build Script

Oct 9, 2026 · @Jaxon Choi

Host makes a party, friends join with a 6-letter code, host taps Pick, everyone sees the same top restaurants. Do one step, test it, then move on. Backend first, then Swift.

How it works

1. **Host** taps Start Party. Server saves a party + makes a code like `K7P2QX`.
2. **Friends** type the code. Server adds them to the party.
3. Everyone's lobby refreshes every 3 seconds and shows who's in.
4. **Host** taps Pick. Server finds places near the party, predicts each member's rating, picks the best for the group, and saves the picks.
5. Everyone's lobby shows the same saved picks.

Before you start

- **Skip Alembic.** Last time I said to make a migration. That's wrong for your project: Alembic isn't hooked up (`env.py` has `target_metadata = None`, and the only migration is empty). `Base.metadata.create_all(...)` at the bottom of `database.py` already makes any **new** table when the server starts.
- **Your app won't build right now.** `Endpoint.partyRecommendations` uses `try` but isn't marked `throws`. Step 7 deletes it. The backend steps don't need the app, so this can wait.

Step 1 — Backend: add the 2 tables

File: `API/database.py`. **Where:** right under `DBWantToGo`, above `Base.metadata.create_all`. **Copy:** `DBWantToGo`. It already has a user foreign key and a `UniqueConstraint`.

Class **`DBParty`**, `__tablename__ = "parties"`

Column	Type	Settings	Why
id	Integer	primary_key, index	row id

Column	Type	Settings	Why
code	String	unique, index, nullable=False	what friends type
host_id	UUID(as_uuid=True)	ForeignKey("users.id"), nullable=False	only the host can Pick
lat	Double	nullable=False	search center
lng	Double	nullable=False	search center
radius	Double	nullable=False	miles, what the phone sends
status	String	nullable=False, default="waiting"	"waiting" then "picked"
picks	String	(allow empty)	results as JSON text, same trick as `restaurants.hours`
created_at	DateTime(timezone=True)	server_default=text("now()")	lets you expire old codes later

Class `DBPartyMember`, `__tablename__ = "party_members"`

Column	Type	Settings	Why
id	Integer	primary_key, index	row id
party_id	Integer	ForeignKey("parties.id"), index, nullable=False	which party
user_id	UUID(as_uuid=True)	ForeignKey("users.id"), index, nullable=False	who joined

Also give it `__table_args__ = (UniqueConstraint("party_id", "user_id"),)` so nobody joins twice.

Imports: add `DateTime` to the `from sqlalchemy import ...` line. Everything else is already imported.

Why `created_at` now: `create_all` never adds columns to a table that already exists. Add it now, even though nothing uses it yet.

Run it: stop the server, then start it again with `python server.py` from the `API` folder. Postgres is already running through Homebrew.

Check it (from the `API` folder):

- `psql -U postgres -h localhost -d app -c "\d parties"`
- `psql -U postgres -h localhost -d app -c "\d party_members"`

Your server uses user `postgres`, password `changeme`, database `app` (the defaults in `database.py`). If psql asks for a password, type `changeme`. Said "role postgres does not exist"? Use `psql app` instead.

Got a column wrong? The tables are empty, so drop them and restart: `psql -U postgres -h localhost -d app -c "DROP TABLE party_members, parties;"`

Done when: both tables show every column above.

Step 2 — Backend: request models

File: `API/models.py`. **Copy:** `WantToGo`.

- `PartyCreate`: `lat: float`, `lng: float`, `radius: float` (miles)
- `PartyJoin`: `code: str`

Pick needs no body. The code goes in the URL.

Done when: the server still starts.

Step 3 — Backend: a shared "nearby restaurants" helper

File: `API/server.py`. **Why:** Pick needs the same search + cache + save that `/find_restaurants` does. Share it, don't copy it.

1. Make `async def _nearby_restaurants(db, lat, lng, radius_miles)` next to `_save_restaurants`.
2. Move the body of `/find_restaurants` into it: from the miles-to-meters line (`* 1609.344`) down to `db.commit()`. Keep the conversion inside, so the cache still matches old searches.
3. It returns the DB restaurant rows: the cached `saved` rows, or the new `db_restaurants`.
4. `/find_restaurants` shrinks to: `await` the helper, then `return _restaurant_list(db, rows, lat, lng)`.

Done when: Find Food in the app works exactly like before.

Step 4 — Backend: fix the group scoring

File: `API/recommendation.py`, class `PartyRecommendation`.

Problem: it averages everyone. If 3 people love a place and 1 hates it, it still wins. **Fix:** "least misery". Throw out places someone would hate, then rank the rest by average.

1. Add a constant at the top with the others: `PARTY_MIN_RATING = 2.5` (below the neutral 3).
2. In `find_restaurants`, for each restaurant, collect every member's predicted rating. Keep the `average` and the `lowest`.
3. Drop places where `lowest < PARTY_MIN_RATING`.
4. If that drops **everything**, keep them all, so it never returns nothing.
5. Sort by average, high to low. Return the top `TOP_PICKS` (already 5) as a list of `(restaurant_id, average, lowest)`.
6. Delete the empty `__init__` (it only says `pass`).

Good to know: `users` = DB user ids (UUIDs). `restaurant_ids` = DB restaurant ids, not Google place ids. Predictions are 1–5.

Done when: a quick test prints sensible results. Set up `db` the way `test_map()` does, then call it with 2 seed users and a few restaurant ids.

Step 5 — Backend: the party endpoints

File: `API/server.py`. Put them where `/party_recommendation` is now, and **delete** that old route.

Imports: `secrets` at the top, `DBParty, DBPartyMember` on the database line, `PartyCreate, PartyJoin` on the models line.

Every route: `current_user: User = Depends(get_current_active_user)` + `db: Session = Depends(get_db)`, then `db_user = _get_db_user(db, current_user)` first.

5a. Helper `_party_info(db, party, db_user)` builds the one JSON shape every party route returns:

```
{
  "code": "K7P2QX",
  "host": "alice",
  "is_host": true,
  "status": "waiting",
  "members": ["alice", "bob"],
  "picks": [{"place_id": "ChIJ...", "name": "Example Cafe", "score": 4.2,
"lowest": 3.6}]
}
```

- `members`: usernames. Join `DBUser` with `DBPartyMember` on the user id, filter by party id, order by `DBPartyMember.id` (join order).
- `host`: the username for `party.host_id`. `is_host`: `party.host_id == db_user.id`.
- `picks`: `json.loads(party.picks)` if set, else `[]`.

5b. Helper `_find_party(db, code)`. `strip()` + `upper()` the code, look it up. Not found: `HTTPException(404, "No party with that code")`.

5c. `POST /party` (body `PartyCreate`)

1. Make a 6-character code with `secrets.choice` from `"ABCDEFGHJKMNPQRSTUVWXYZ23456789"`. No 0/O or 1/I/L, so it's easy to read out loud.
2. Code already taken? Make another (loop).
3. Add `DBParty(...)`, then `db.flush()` to get its id. Same trick `find_restaurants` uses on `db_query`.
4. Add the host as a `DBPartyMember`. `db.commit()`.
5. Return `_party_info(...)`.

5d. `POST /party/join` (body `PartyJoin`)

1. `_find_party`.
2. Add a `DBPartyMember`. On `IntegrityError`: `db.rollback()`, since they're already in. Same as `add_want_to_go`.
3. Return `_party_info(...)`.

5e. `GET /party/{code}`

1. `_find_party`.
2. Not a member: `HTTPException(403, "Join the party first")`.
3. Return `_party_info(...)`.

5f. `POST /party/{code}/pick`

1. `_find_party`. Not the host: `HTTPException(403, "Only the host can pick")`.
2. `rows = await _nearby_restaurants(db, party.lat, party.lng, party.radius)`. No rows: `HTTPException(400, "No restaurants nearby. Try a bigger radius.")`.
3. Get the member user ids. Call `PartyRecommendation().find_restaurants(db, member_ids, [r.id for r in rows])`. Note the `()`.
4. Build picks as `{"place_id", "name", "score", "lowest"}`. Round numbers to 1 decimal like `/recommend` does.
5. `party.picks = json.dumps(picks)`, `party.status = "picked"`, `db.commit()`.
6. Return `_party_info(...)`.

Done when: the server starts and the 4 routes show up at `localhost:8000/docs`.

Step 6 — Backend: test with curl

Why curl: the Authorize button in `/docs` won't work. `tokenUrl="token"` in `auth.py` points at a route you don't have.

Setup: register `alice`, `bob` and `carol` once (`POST /auth/register`). Log each in and save their `access_token`, e.g. `A=...`, `B=...`, `C=...` in your terminal. Got a 401 later? Log in again.

```
curl -s -X POST localhost:8000/auth/login -H "Content-Type: application/json"
-d '{"username":"alice","password":"YOUR_PASSWORD"}'
```

Templates (swap the token, path and body):

```
curl -s -X POST localhost:8000/party -H "Authorization: Bearer $A" -H
"Content-Type: application/json" -d
'{"lat":37.7749,"lng":-122.4194,"radius":1}'
curl -s localhost:8000/party/CODE -H "Authorization: Bearer $B"
```

Checklist:

- [ ] Alice `POST /party`: a code, `status` "waiting", `members` ["alice"]
- [ ] Bob `POST /party/join` with the code: `members` ["alice","bob"], `is_host` false
- [ ] Bob joins again: still 2 members, no error
- [ ] Bob `GET /party/CODE`: same as above

- [ ] Bob `POST /party/CODE/pick`: 403
- [ ] Alice `POST /party/CODE/pick`: `status` "picked", up to 5 picks
- [ ] Bob `GET /party/CODE`: the same picks
- [ ] Bob joins `ZZZZZZ`: 404
- [ ] Carol `GET /party/CODE` without joining: 403

Done when: every box is ticked. Don't start Swift until it is.

Step 7 — Swift: delete the old party stubs

Why: they send a list of users from the phone. Any phone could add anyone. The server now reads members from the table.

Delete:

- `PartyRecommendationDTO` and `PartyRecommendationPayload` in `DTOs.swift`
- `partyRecommendations(users:restaurants:)` in `Endpoint.swift` (this is the one breaking the build)
- `partyRecommendations(users:restaurants:)` in `FunctionManager.swift`

Done when: the app builds.

Step 8 — Swift: DTOs

File: `DTOs.swift`. **Copy:** `WantToGoDTO` and `WantToGoPayload`.

Struct	Fields	Conforms to
PartyPickDTO	placeId: String, name: String, score: Double, lowest: Double, `var id: String { placeId }`	Decodable, Identifiable
PartyDTO	code: String, host: String, isHost: Bool, status: String, members: [String], picks: [PartyPickDTO]	Decodable
CreatePartyPayload	lat: Double, lng: Double, radius: Double	Encodable
JoinPartyPayload	code: String	Encodable

Contract check. Your `JSONDecoder.api` turns snake_case into camelCase:

- `is_host` = `isHost`
- `place_id` = `placeId`
- Everything else has the same name on both sides.

Done when: the app builds.

Step 9 — Swift: Endpoints

File: `Endpoint.swift`. **Copy:** `addWantToGo` (POST with a body) and `wantToGo` (GET). All `requiresAuth: true`.

Function	Method	Path	Body	`throws`?
createParty(lat:lng:radius:)	POST	/party	CreatePartyPayload	yes
joinParty(code:)	POST	/party/join	JoinPartyPayload	yes
party(code:)	GET	/party/(code)	none	no
pickParty(code:)	POST	/party/(code)/pick	none	no

Rule: uses `try` inside means mark it `throws`. That's the bug from Step 7.

Step 10 — Swift: PartyManager

New file: `PartyManager.swift`. **Copy:** the top of `FunctionManager` (`@Observable`, `@MainActor`, `final class`, `apiClient`, `init`).

- State: `var party: PartyDTO?` (nil means not in a party)
- `create(lat:lng:radius:)`: send `createParty`, save the result into `party`
- `join(code:)`: send `joinParty`, save into `party`
- `refresh()`: if `party` is nil, return. Else send `party(code:)` and save
- `pick()`: send `pickParty`, save
- `leave()`: set `party = nil` (only on the phone for now)

All `async throws`, except `leave()`.

Wire it up, or the app crashes when a view asks for it:

1. `Restaurant_RecommenderApp.swift`: add `@State private var partyManager: PartyManager`, create it in `init` with the **same** `apiClient`, add `.environment(partyManager)`.
2. `RootView` `#Preview`: add `.environment(PartyManager(apiClient: apiClient))`.

Done when: the app builds and runs like before.

Step 11 — Swift: the views

Same pattern as your other views: `@State private var errorMessage`, `isSubmitting`, `defer { isSubmitting = false }`, errorMessage shown in red, `.disabled(isSubmitting)` on buttons.

11a. `PartyHomeView`: start or join

- `@Environment(PartyManager.self)` and `@State private var locationManager = LocationModel()`
- Section "Start a party": radius `Slider` (1 to 10 miles, step 1) + **Start Party** button. Get the spot with `await locationManager.currentLocation()`, same as Find Food. Nil: show "Allow location access". Then `partyManager.create(...)`.
- Section "Join a party": `TextField("Code")` with `.textInputAutocapitalization(.characters)` and `.autocorrectionDisabled()`. **Join** button disabled unless the code has 6 characters.
- On success, go to the lobby: a `showLobby` bool + `.navigationDestination(isPresented:)`.

11b. `PartyLobbyView`: the waiting room

- The code, big: `.font(.largeTitle.monospaced())`
- `ShareLink(item: "Join my food party! Code: \(code)")`
- A list of `members`
- Host and status "waiting": **Pick Restaurant** button that calls `partyManager.pick()`
- Status "picked": show `PartyResultsView`
- **Auto-refresh:** `.task { }` with a loop: `refresh()`, then `Task.sleep(for: .seconds(3))`. Stop when status is "picked". `.task` stops by itself when you leave the screen.
- **Leave** toolbar button: `partyManager.leave()`, then `dismiss()`

11c. `PartyResultsView`: the winners

- `List(picks)`: name, "Group 4.2", "Lowest 3.6" (`specifier: "%.1f"`), and your `StarRatingView(rating: score.rounded())`
- First row is the winner: `Label("Winner", systemImage: "trophy")`
- Bonus: tap a row to open `ModalContentView`, like `WantToGoView` does

11d. Menu link. In `PostLoginView`, add `NavigationLink(destination: PartyHomeView())` with `Label("Party", systemImage: "person.3")`.

Done when: you can start a party and see your own code in the lobby.

Step 12 — Full test: app + curl

- [ ] Backend up. Run the app in the Simulator as **alice**. Party, then Start Party. Note the code.
- [ ] Terminal as **bob**: `POST /party/join` with that code. Bob shows up in alice's lobby within 3 seconds.
- [ ] Alice taps Pick. Results show.
- [ ] Bob `GET /party/CODE`: the same picks.
- [ ] Swap: bob creates with curl, alice joins in the app, bob picks with curl. Alice's lobby shows the picks.
- [ ] Wrong code in the app: red error message.
- [ ] Location denied: red error message.

Done when: every box is ticked. The feature works.

Later (not now)

- **Faster Pick:** `recommend()` reloads every review once per member. Load `load_ratings` and `load_onboarding` once, then call `predict()` for each member.
- **Open now + distance:** reuse `context_adjustment()` to rank group picks.
- **Expire codes** after 24 hours using `created_at`.
- **Leave on the server:** `DELETE /party/{code}/members/me`.
- **Live updates:** WebSockets instead of refreshing every 3 seconds.
- **Vote:** members thumbs-up the top 3 before the final pick.

Links

For step	Link
1	SQLAlchemy ORM quickstart (Column, ForeignKey)
1	SQLAlchemy UniqueConstraint
2, 5	FastAPI request body
5	FastAPI path parameters
5	FastAPI errors (HTTPException)
5	Python secrets
4	Group recommender strategies, incl. least misery (Masthoff)
11	SwiftUI ShareLink
11	SwiftUI .task
11	navigationDestination(isPresented:)
Later	FastAPI WebSockets